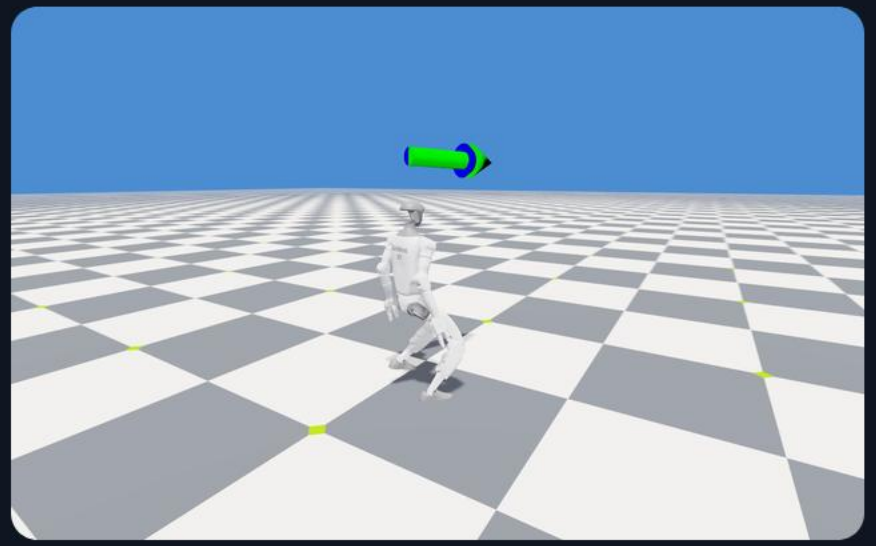
ENGINEERING WORKSHOP · ISAAC LAB 3 + NEWTON

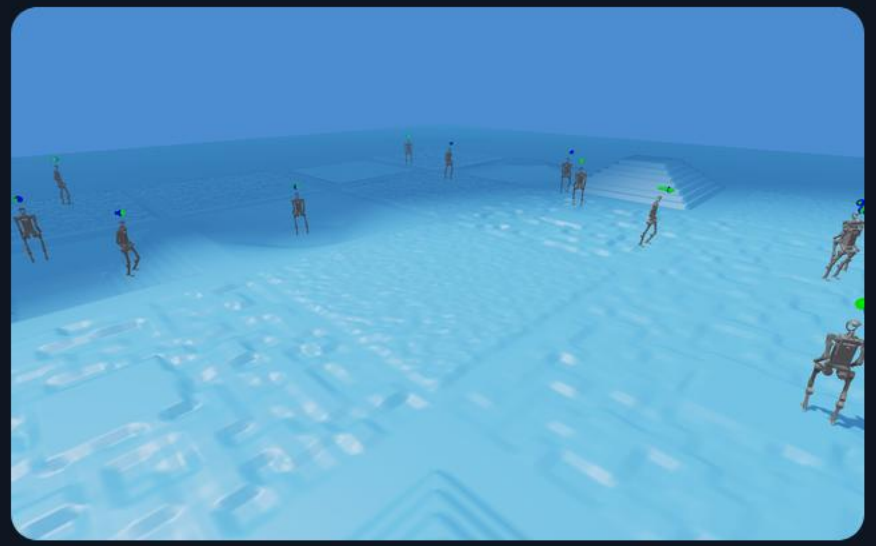
Simulation for Humanoid

Week 3 — Humanoid Robots

Isaac Lab 3.0 + Newton 2026 edition

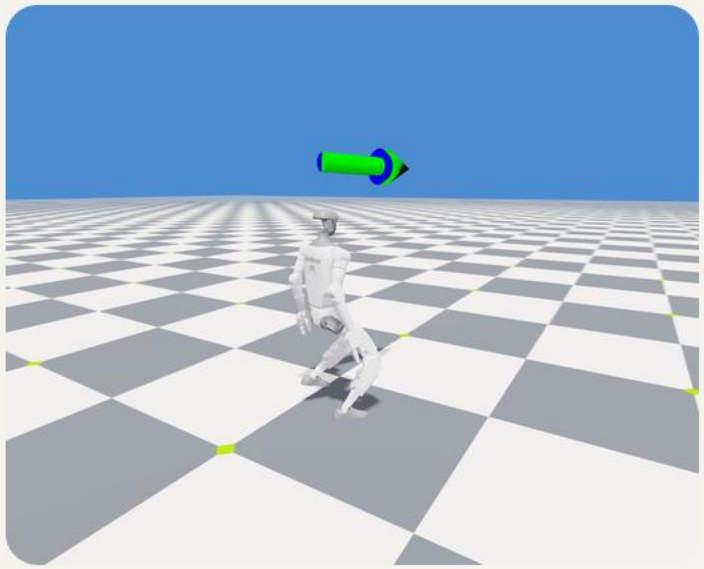
Pannatorn Bongkotamphon
Dr. Sukhum Sattaratnamai





WEEK 3

Agenda



01 Recap Week 2

02 Meet the humanoids · H1 · G1 · Digit

03 It’s all in the hips (F-A-R)

04 Pretrained policy · H1 in a warehouse

05 ขับ G1 ด้วยคีย์บอร์ด (Newton)

06 Exercises

RECAP FROM WEEK 2

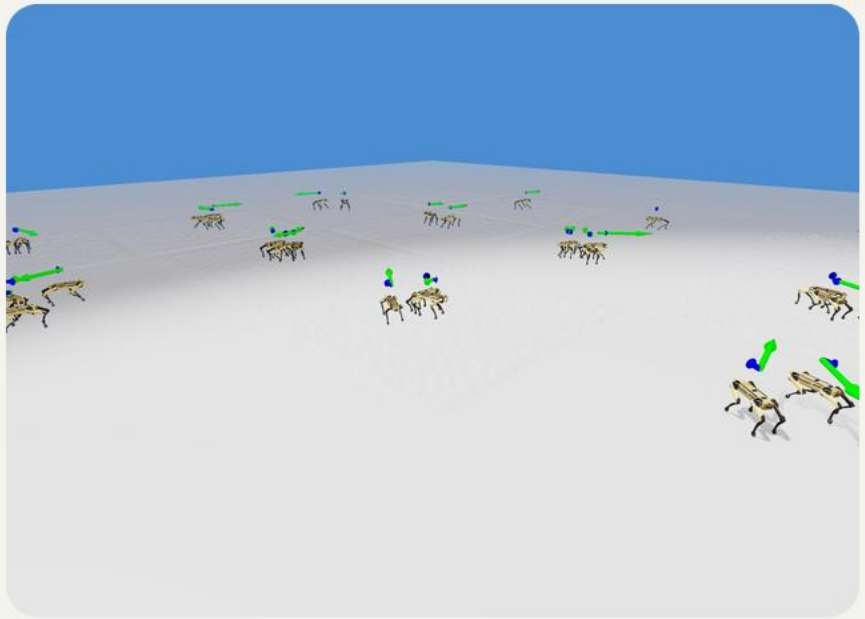
Week 2 ได้อะไรไปแล้ว

TRAIN

```
uv run isaaclab train --rl_library rsl_rl \
  --task IsaacContrib-Velocity-Flat-Spot \
  physics=newton_mjwarp
```

PLAY

```
uv run isaaclab play --rl_library rsl_rl \
  --task IsaacContrib-Velocity-Flat-Spot \
  --checkpoint latest --viz newton \
  physics=newton_mjwarp
```

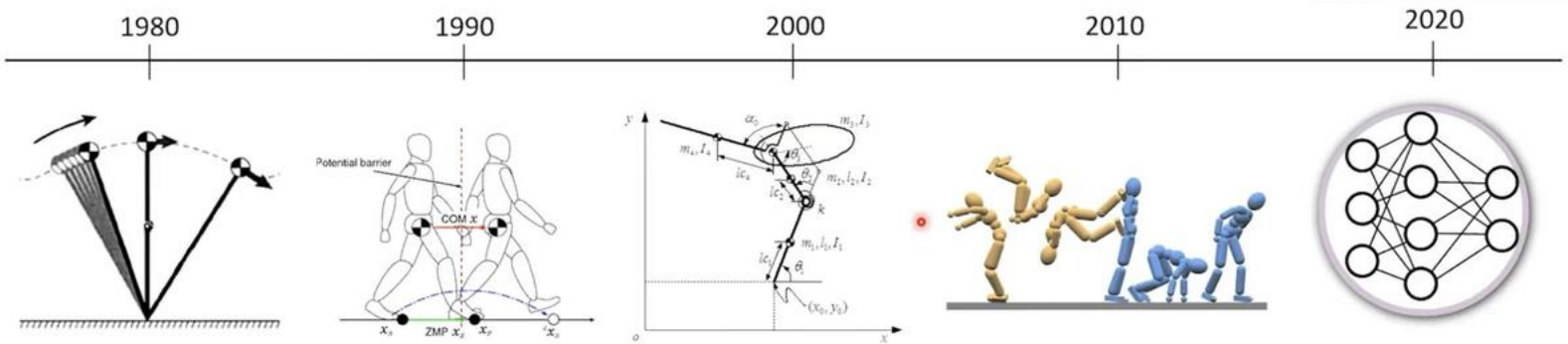


ผ่าน = Spot เดินตามลูกศรเขียวได้

HOW WE GOT HERE

วิธีคุมหุ่นสองขา 1980 → 2020s

workshop นี้ = ขวาสุด: Deep RL



- Simple
- Only works on simple scenarios

- Simple
- No theoretical foundations
- Too restrictive

- Relatively accurate model
- Does not produce new behaviors

- Generate new behaviors
- Complex and often not scalable
- inaccurate models

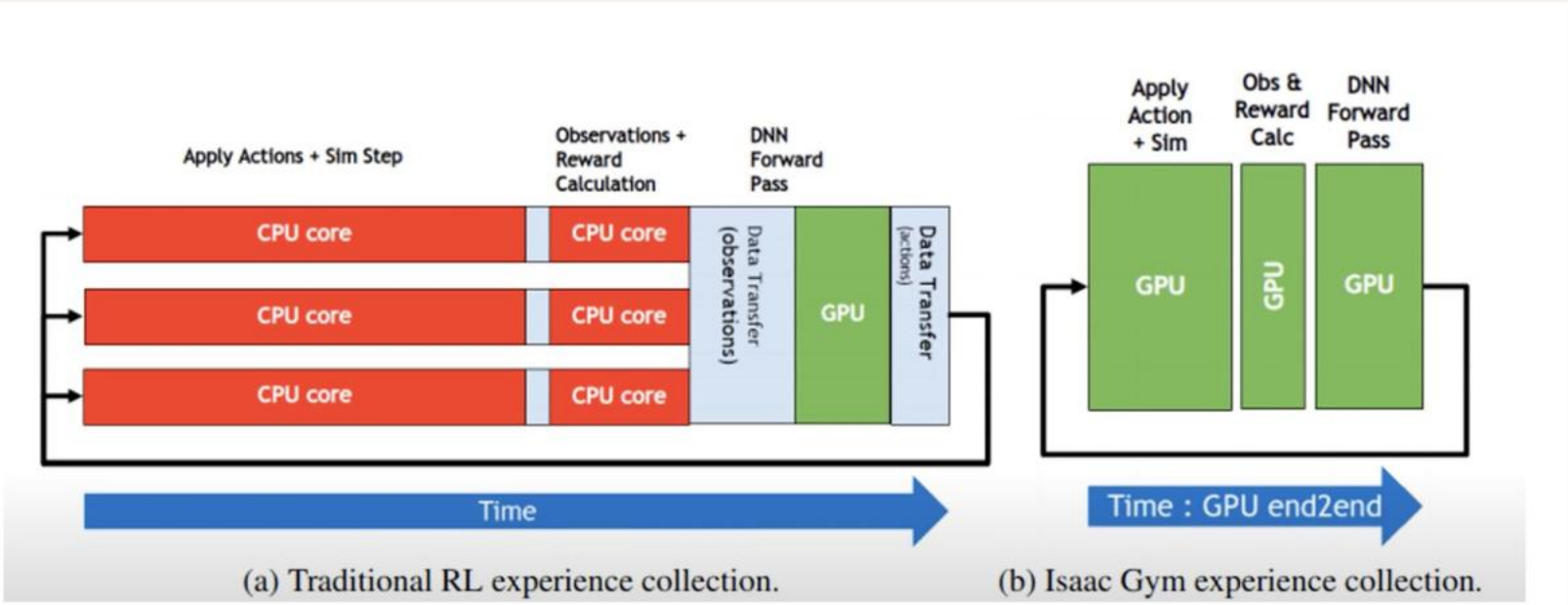
- Scalable
- Accurate models
- Discovers new behaviors
- Lightweight policy
- Too slow??
- Too much tuning

อ้างอิง : https://www.materic.or.kr/

ภาพ: materic.or.kr

WHY GPU SIMULATION CHANGED EVERYTHING

ทุกอย่างอยู่บน GPU ตั้งแต่ต้นจนจบ



2021

Isaac Gym เริ่มแนวคิดนี้

2026

Newton บน Warp ทำต่อ

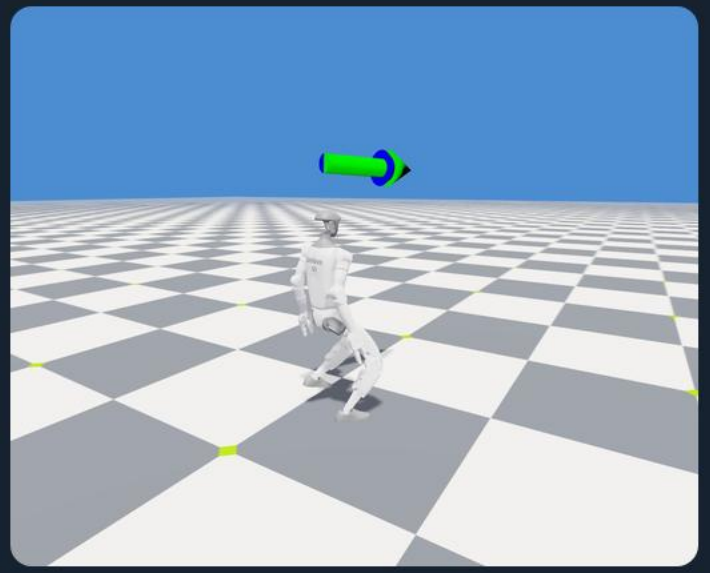
Spot 2,048 ตัวบนโน้ตบุ๊กเรา

≈ 21k steps/s

ภาพ: arxiv.org/pdf/2109.11978 (Learning to Walk in Minutes Using Massively Parallel Deep RL)

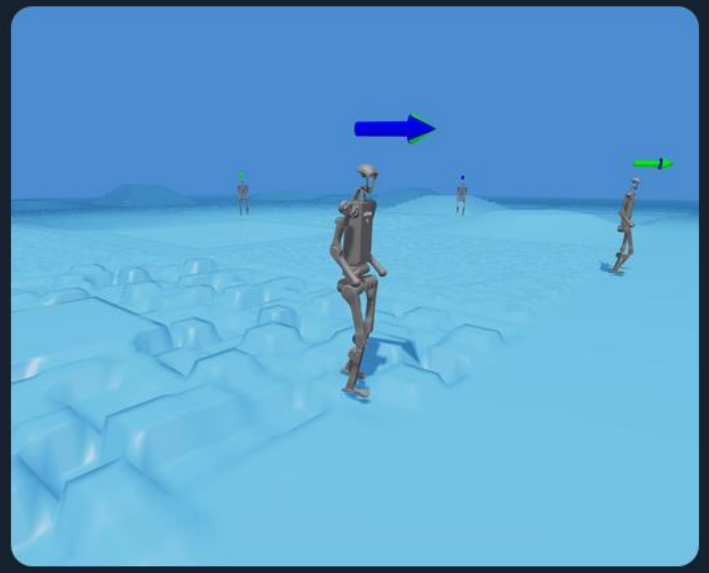
LET'S MEET · HUMANOIDS IN ISAAC LAB 3

หุ่นสองขาที่ใช้ใน workshop



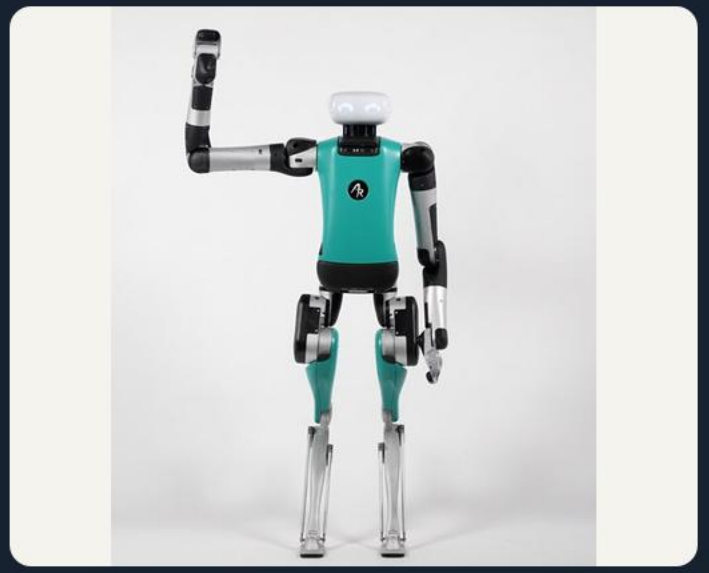
Unitree G1

`Isaac-Velocity-Flat-G1`
`Isaac-Velocity-Rough-G1`



Unitree H1

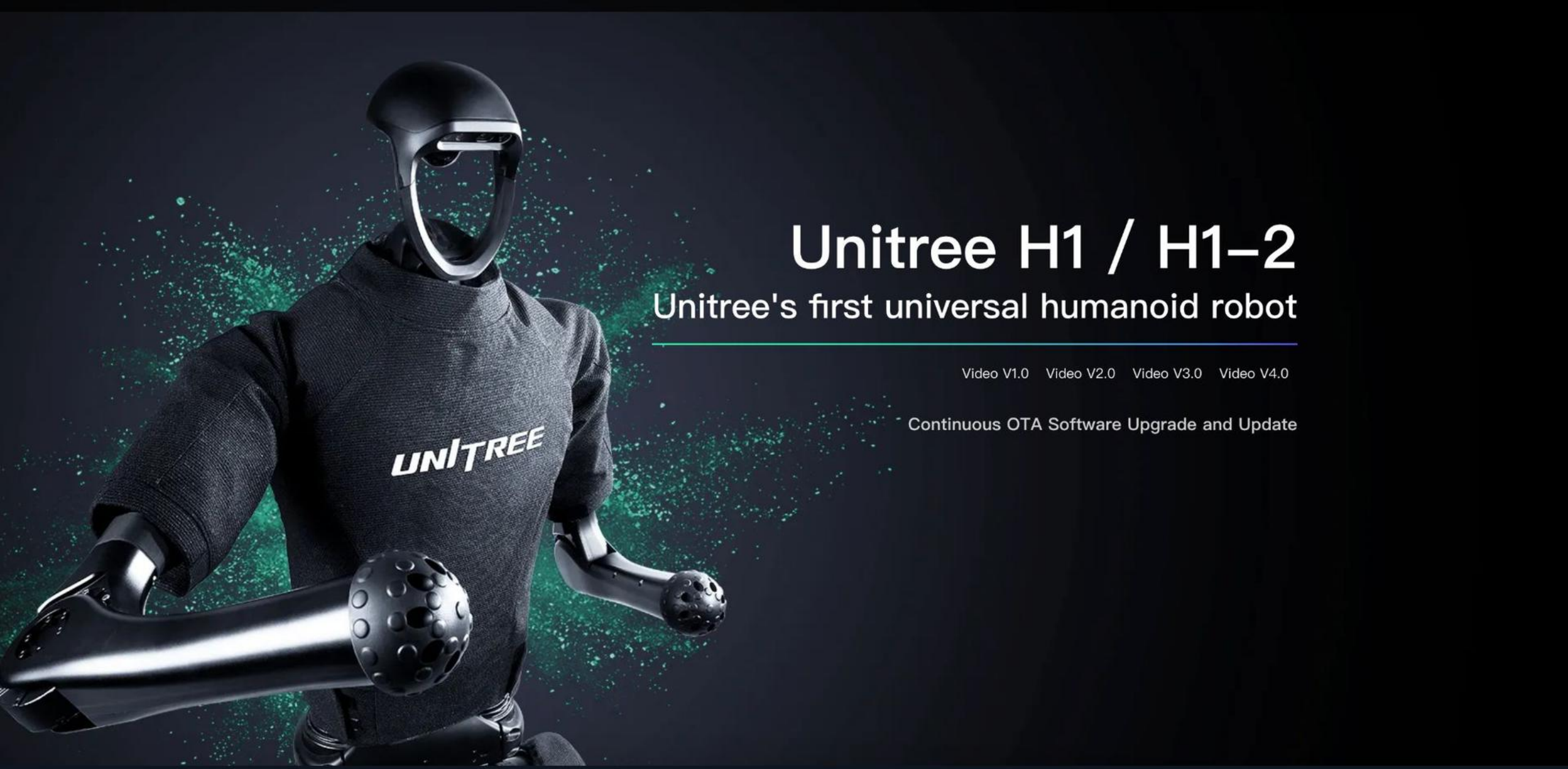
`Isaac-Velocity-Flat-H1`
`Isaac-Velocity-Rough-H1`



Agility Digit

`IsaacContrib-Velocity-Flat-Digit`
`IsaacContrib-Velocity-Rough-Digit`

ดูทั้งหมด: uv run isaaclab list_envs --keyword Velocity · ไม่มี -v0 แล้ว · G1/H1 ใช้ Newton เป็นค่าเริ่มต้น



Unitree H1 / H1–2

Unitree's first universal humanoid robot

Video V1.0 Video V2.0 Video V3.0 Video V4.0

Continuous OTA Software Upgrade and Update

Unitree H1

`Isaac-Velocity-Flat-H1 · Rough-H1`

ภาพ: unitree.com/H1

UNITREE H1 · ACTUATORS

หัวใจของหุ่นคือมอเตอร์ข้อต่อ

M107 PK International mainstream joint motor

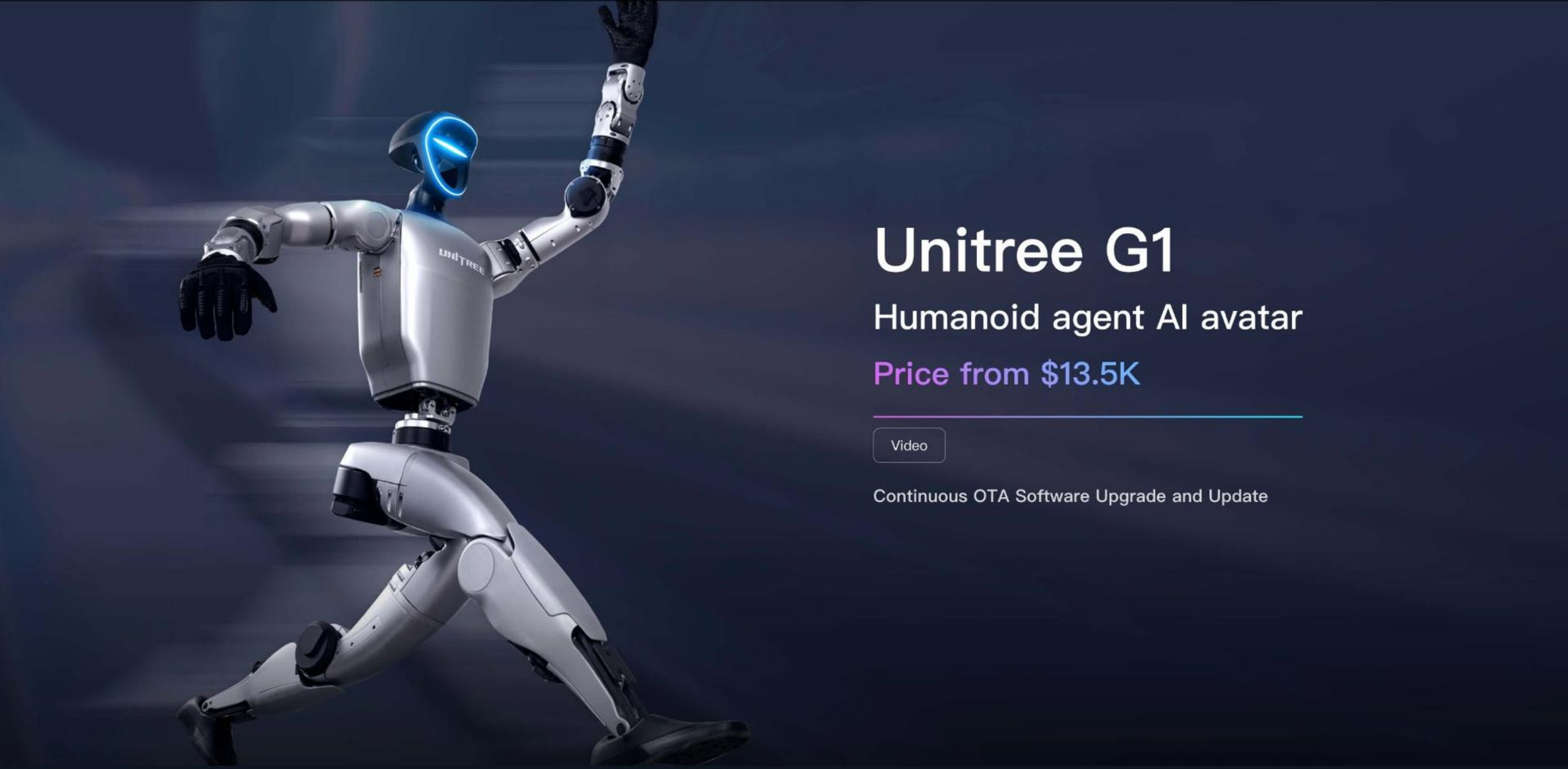
Product	 M107	 T-1	 T-2
Maximum torque OR pulling force (3.5cm arm equivalent)	360N.m 10000N	180N.m	8000N
Weight	1.9kg	2.26kg	2.2kg
Maximum Torque or Tension/Weight Ratio	189 5263	79	3636
Hollow shaft	YES	YES	-
Dual Encoder	YES	YES	YES
Dimensions(mm)	107 × 74	100 × 130	60 × 180

ภาพ: unitree.com/H1

ใน sim มอเตอร์ = actuator model

stiffness · damping · effort limit · velocity limit

> ค่าอยู่ใน isaaclab_assets/robots/unitree.py — Week 8 เราจะปรับเอง



Unitree G1

Humanoid agent AI avatar

Price from $13.5K

Video

Continuous OTA Software Upgrade and Update

UNITREE G1 · SPEC

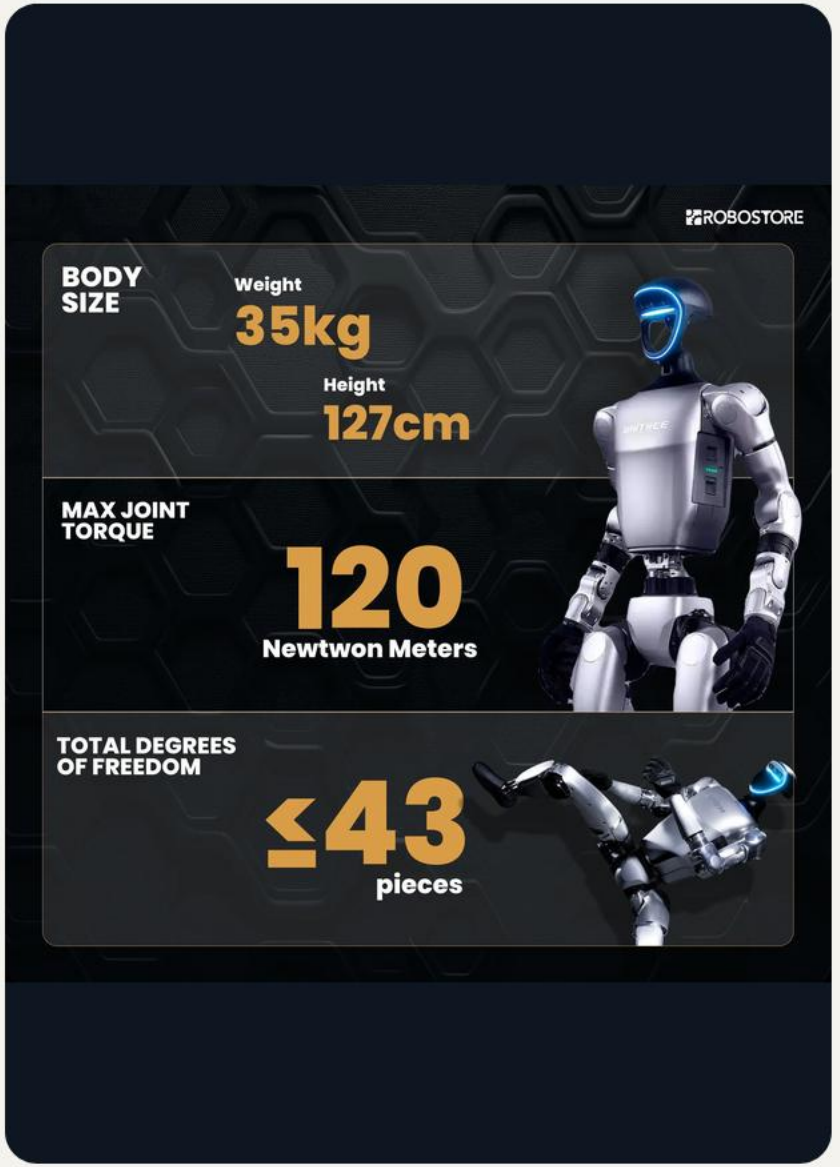
เล็ก เบา คล่อง

35 kg
น้ำหนัก

127 cm
ความสูง

120 N·m
torque สูงสุดที่ข้อต่อ

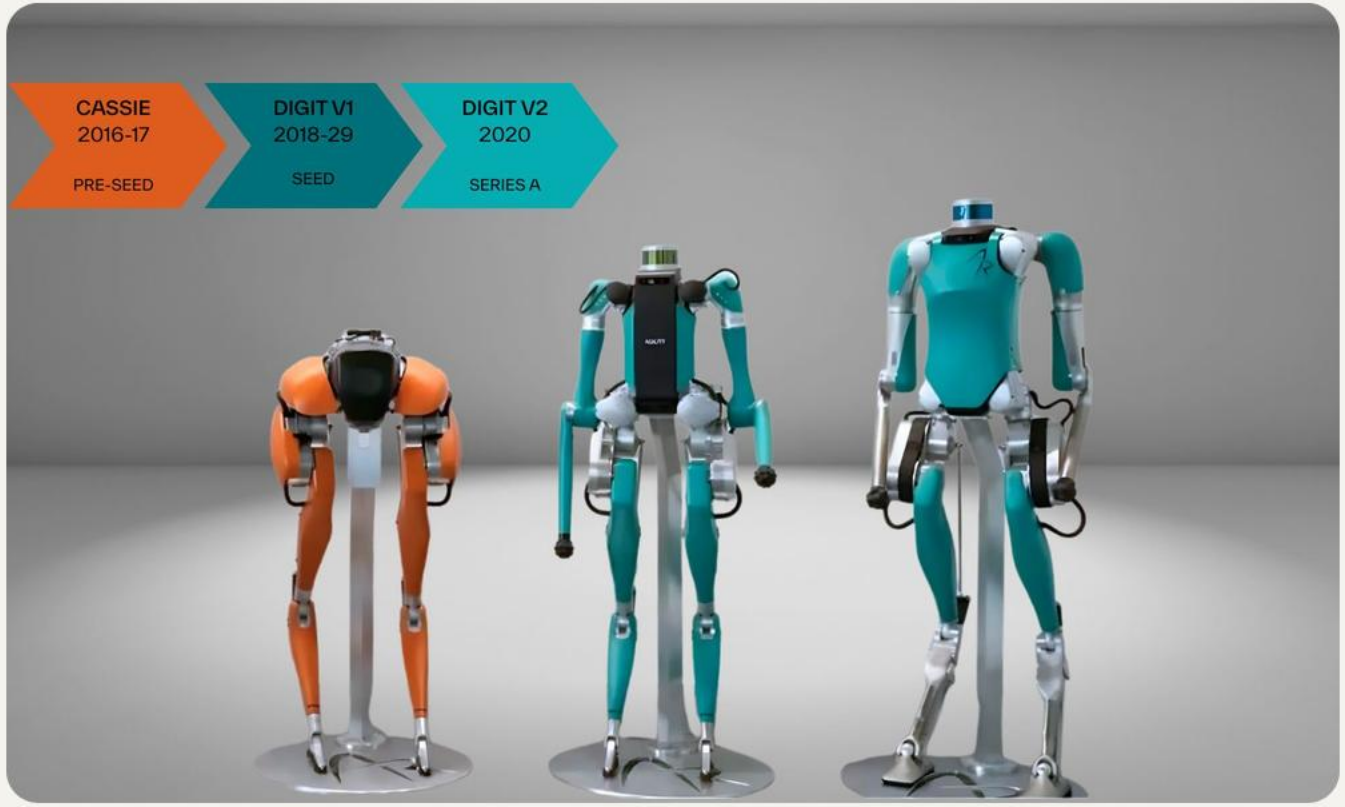
≤ 43
DoF (รุ่นท็อป)

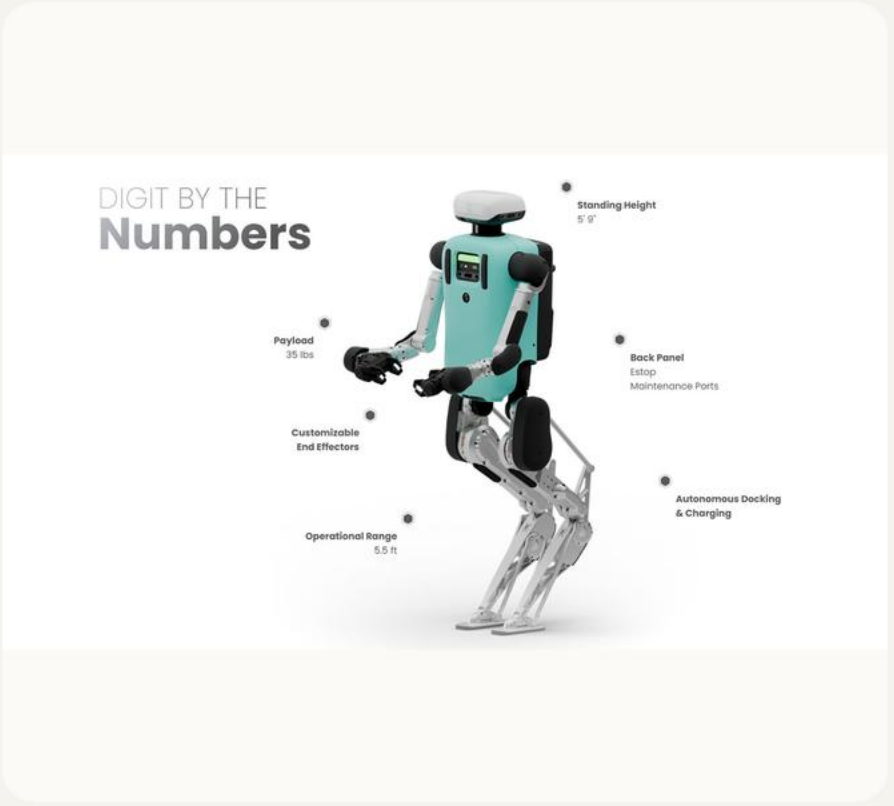


ภาพ/สเปก: robostore.com (Unitree G1 EDU vs H1-2 comparison)

AGILITY ROBOTICS · DIGIT

จาก Cassie → Digit





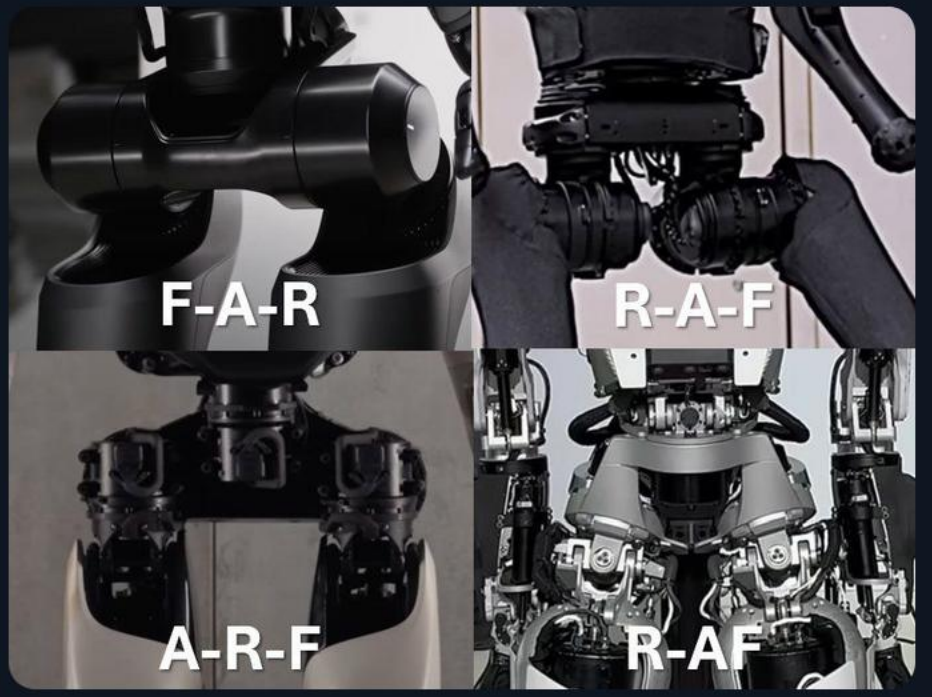
ใน Isaac Lab: **Isaac-Velocity-*-Cassie** และ **IsaacContrib-Velocity-*-Digit**

ภาพ: agilityrobotics.com · xpert.digital

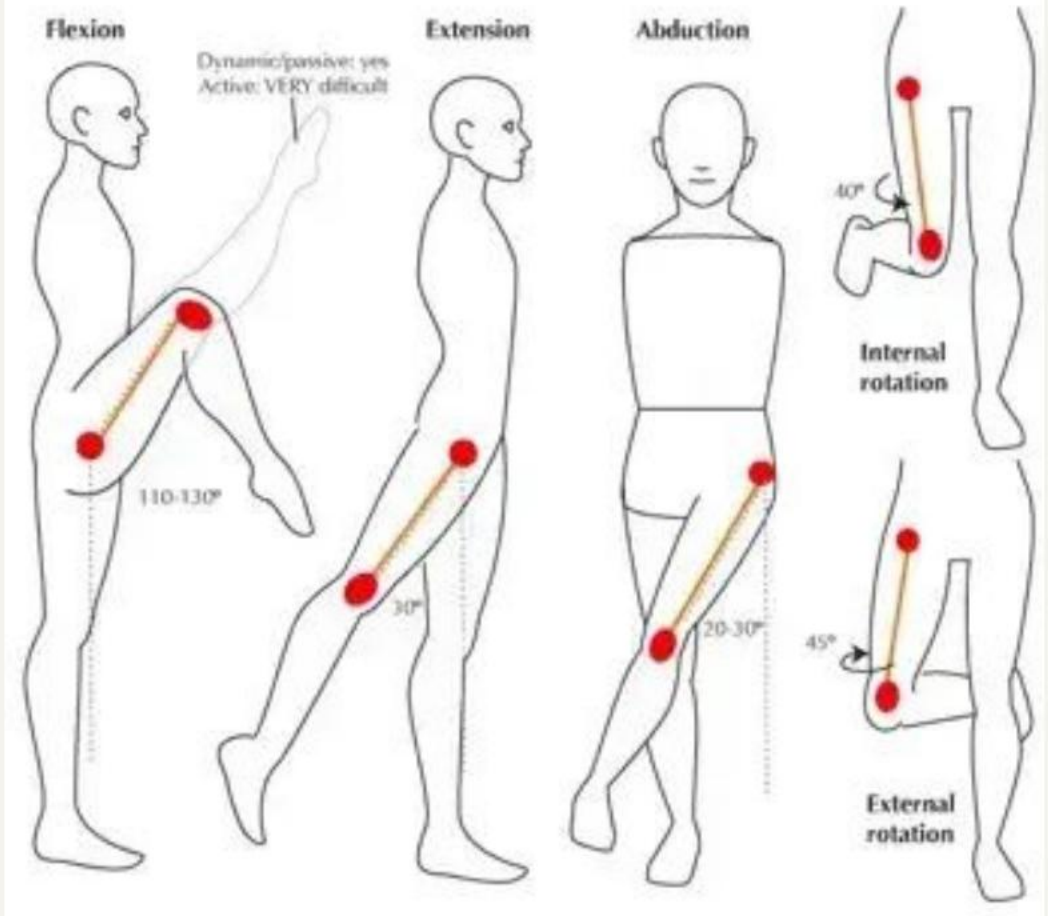
ROBOT DESIGN

It’s all in the hips

ทำไมหุ่นสองขาเกือบทุกค่ายถึงหันมาใช้สะโพกแบบ F-A-R



ที่มา: humanoidsdaily.com — It’s all in the hips: why humanoid robot designs are converging on a specific joint configuration



3 ทิศของข้อสะโพก

F-A-R คืออะไร?

F · Flexion / Extension

ขาไปหน้า–ถอยหลัง · ใช้ตอนก้าวเดิน/วิ่ง

A · Abduction / Adduction

กางขาออก–หุบเข้า · สำคัญมากกับการทรงตัว

R · Rotation

บิดขา หมุนปลายเท้าเข้า–ออก · ใช้ตอนเลี้ยว

F-A-R = ลำดับข้อต่อสะโพก จากบนลงล่าง

A-R-F → F-A-R

ทำไมถึงเปลี่ยน

รุ่นแรกนิยม A-R-F

มอเตอร์ F ต้องแรงมาก → เอาไว้ล่างสุด จะได้ไม่ต้องแบกน้ำหนักมอเตอร์ตัวอื่นตอนแกว่งขา

แต่ข้อเสีย

- จุดหมุนขาอยู่ต่ำ → ก้าวสั้น
- ดู “ขาสั้น ลำตัวยาว”
- จัดวางกลไกยุ่งยาก

F-A-R

ยกจุดหมุน F ขึ้นสูง → ก้าวยาว สัดส่วนเหมือนคน — มอเตอร์ยุคนี้เล็กลงแต่แรงขึ้น จึงแบกได้สบาย

ที่มา: humanoidsdaily.com



Tesla Optimus ยังใช้ A-R-F

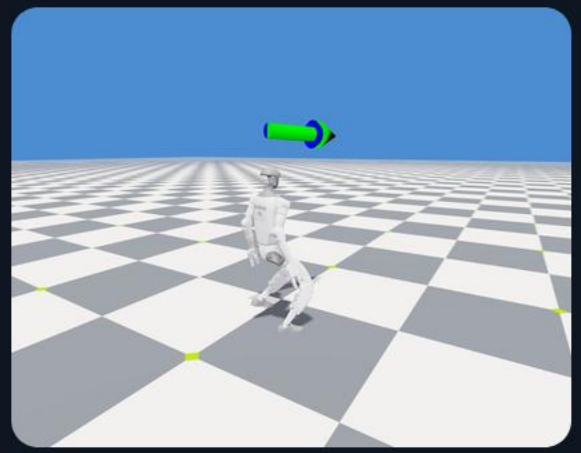
WHY F-A-R? · เหตุผลเชิงออกแบบที่มักถูกยกมา

ทำไม G1 และอีกหลายรุ่นเลือก F-A-R

ใช้ F-A-R

Figure 02 · Unitree G1 · PNDbotics

G1 ตรวจเองจากไฟล์ · ที่เหลือตามบทความ Humanoids Daily



ก้าวได้ยาวขึ้น

จุดหมุนหน้า–หลัง (F) อยู่สูง → stride ยาวกว่า

สัดส่วนเหมือนคนกว่า

ลำตัวยาวขึ้น ขาไม่ดูสั้น

จัดกลไกสวย

มอเตอร์ R ซ่อนในต้นขา · F อยู่สูง · A คั่นกลาง

ประกอบง่ายกว่า

เดินสายไฟสะดวกขึ้น

ข้อแลกเปลี่ยน: F ต้องแกว่งมอเตอร์ A และ R ไปด้วยทุกก้าว — มอเตอร์ที่เล็กแต่แรงขึ้นทำให้รับได้

WHO USES WHAT

ไม่ได้มีแค่ F-A-R

ตรวจเองจากไฟล์ใน ISAAC LAB 3

หุ่น	จากเชิงกรานลงไป	ลำดับ
Cassie (2017)	abduct → rotate → flex	**A-R-F**
H1 (2023)	yaw → roll → pitch	**R-A-F**
G1 (2024)	pitch → roll → yaw	**F-A-R**
poclegs	HR → HAA → HFE	**R-A-F**

สร้าง env แล้วพิมพ์ joint parent → child ของขาซ้าย · ทดสอบ 10 ต.ค. 2026

ตามบทความ HUMANOIDS DAILY (เม.ย. 2025)

F-A-R · Figure 02, Unitree G1, PNDbotics

A-R-F · Tesla Optimus ทุกรุ่น

parallel R-A-F · XPeng

ไม่ยืนยัน · 1X (cable-driven, บทความแค่คาดว่าใกล้ F-A-R)

แทบไม่มีใครใช้ · R-F-A, A-F-R

ที่มา: humanoidsdaily.com “It’s all in the hips” (14 เม.ย. 2025, วิเคราะห์โดย Scott Walter) · บทความระบุ “Sanctuary AI (Apollo)” ด้วย แต่ Apollo เป็นหุ่นของ Apptronik จึงไม่ใส่

สรุป

ไม่มีลำดับเดียวที่ถูก — เลือกตามสิ่งที่ให้ความสำคัญ

A-R-F

Cassie · 2017

→

R-A-F

H1 · 2023 · poclegs

→

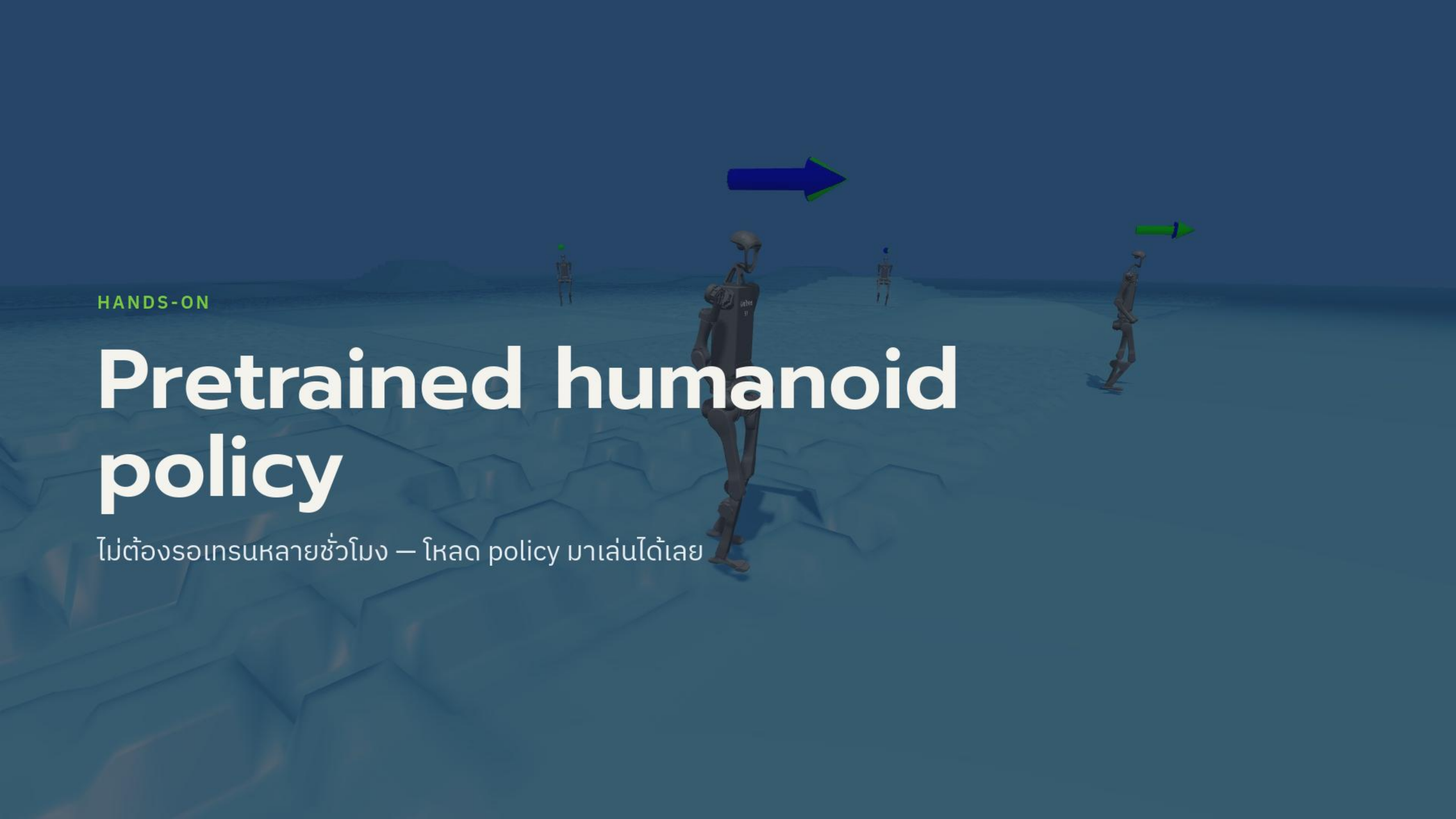
F-A-R

G1 · 2024

ข้อที่อยู่**บนสุด**ต้องหมุนมอเตอร์ของข้อที่อยู่ใต้มันไปด้วย · ข้อ**ล่างสุด**หมุนได้เบาที่สุด

F-A-R: แกนก้าวขาอยู่สูงและตรง ก้าวยาว แต่ F ต้องแกว่งน้ำหนักมากขึ้น · R-A-F: ข้อก้าวขา (F) อยู่ล่างสุด แกว่งเบา แต่แกนก้าวจะเอียงตาม R และ A

และยังไม่จบ: สะโพกทำงานคู่กับเอว (waist) และไหล่ ซึ่งยังมีการออกแบบหลากหลายกว่าสะโพก



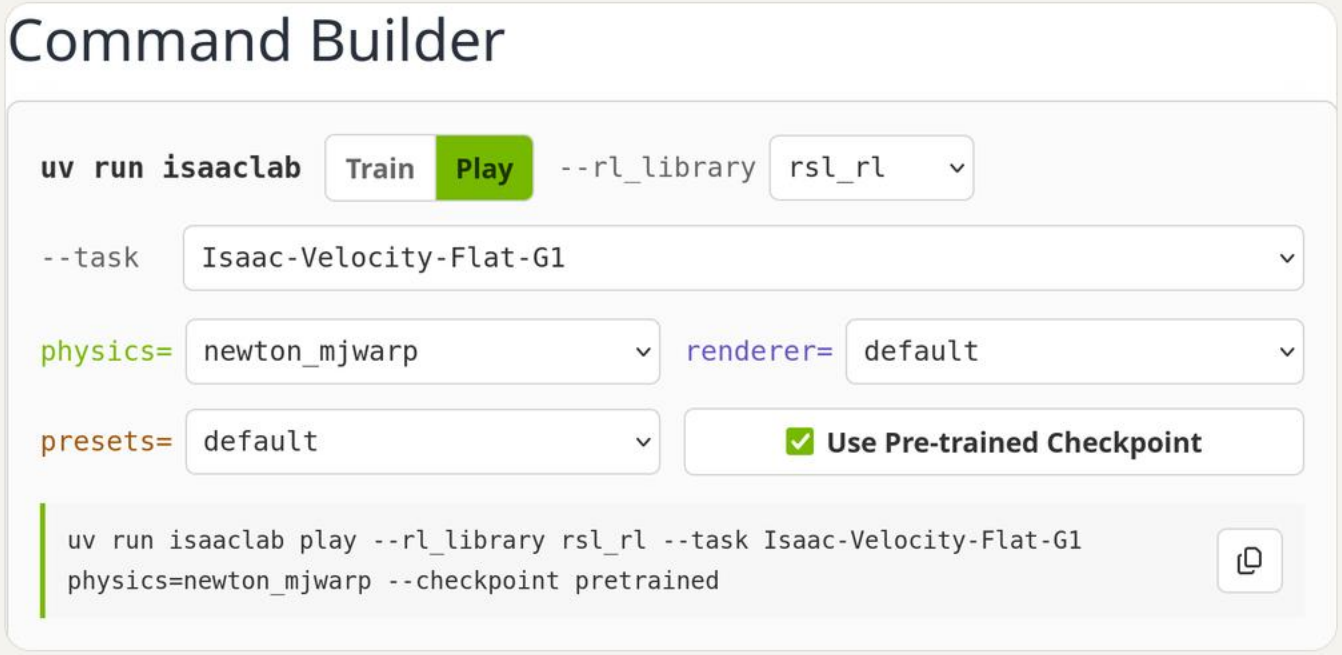
HANDS-ON

Pretrained humanoid policy

ไม่ต้องรอเทรนหลายชั่วโมง — โหลด policy มาเล่นได้เลย

FROM THE DOCS · COMMAND BUILDER

ติ๊ก Pre-trained แล้วก๊อปได้เลย




```
… physics=newton_mjwarp --checkpoint pretrained --viz newton
```

1 **เลือก Play**

rsl_rl

2 **--task Isaac-Velocity-Flat-G1**

หรือ Rough-G1 · Flat-H1 · Rough-H1

3 **ติ๊ก Use Pre-trained Checkpoint**

ได้ --checkpoint pretrained ต่อท้าย

4 **ก๊อป แล้วเติม --viz newton**

builder ไม่ใส่ viewer ให้ — ไม่เติมจะรันแบบ headless

คำสั่งที่ได้ทดสอบแล้วบนเครื่องนี้ (หน้า Pretrained ถัดไป) · ภาพ: isaac-sim.github.io/IsaacLab/v3.0.0-EA/source/setup/environments.html

NEW IN ISAAC LAB 3.0

--checkpoint pretrained

```
$ uv run isaaclab play --rl_library rsl_rl \
    --task Isaac-Velocity-Flat-G1 --num_envs 36 \
    --checkpoint pretrained --viz newton

Fetching pre-trained checkpoint : …/rsl_rl/Isaac-Velocity-Flat-G1_newtonmjwarp_none_rsl_rl.pt
```

ตรง backend

โหลด policy ที่เทรนบน Newton ให้อัตโนมัติ

โหลดครั้งเดียว

เก็บไว้ใน ~/IsaacLab/.pretrained_checkpoints/

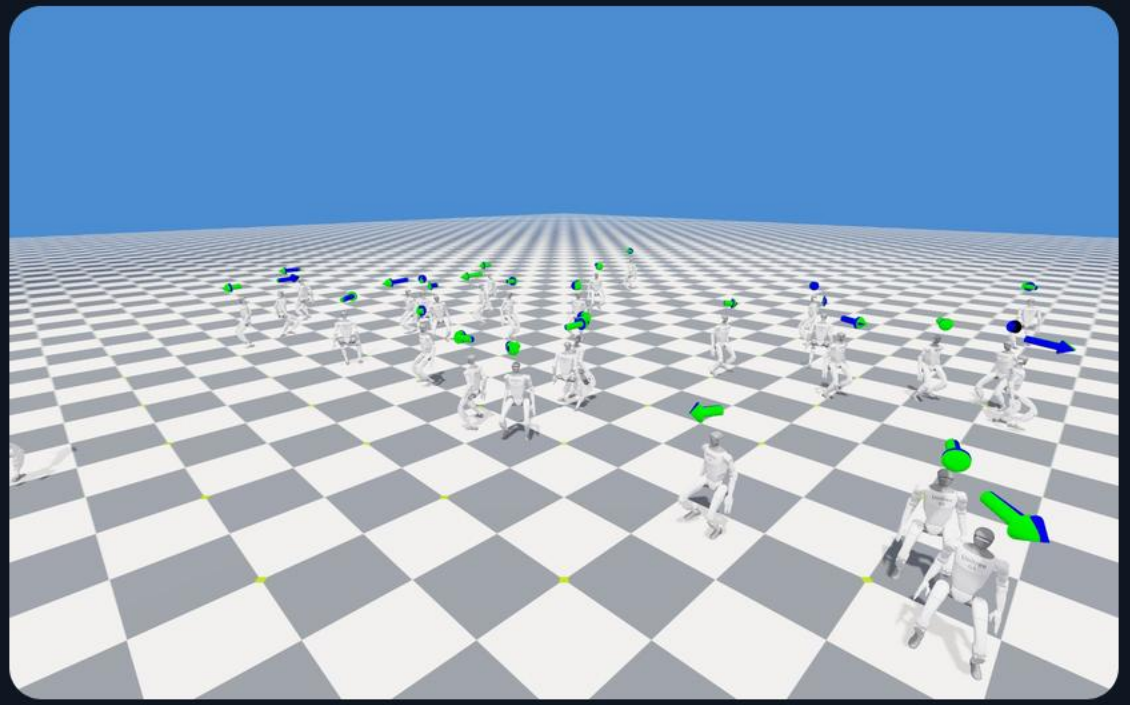
ทดสอบแล้ว

G1 Flat และ H1 Rough ใช้ได้ทั้งคู่

แทน --use_pretrained_checkpoint แบบเดิม

RESULT · REAL SCREENSHOTS

หุ่นเดินตามคำสั่งของตัวเอง



G1 Flat · 36 envs · pretrained



H1 Rough · pretrained

ลูกศรเขียว = ความเร็วที่สั่ง (สุ่มเองทุกตัว) · ลูกศรน้ำเงิน = ความเร็วจริง

TRAIN IT YOURSELF

เทรน H1 บน rough terrain เอง

```
$ uv run isaaclab train --rl_library rsl_rl --task Isaac-Velocity-Rough-H1
$ uv run isaaclab play  --rl_library rsl_rl --task Isaac-Velocity-Rough-H1 \
      --num_envs 32 --checkpoint latest --viz newton
```

48 s

1,024 envs × 60 iterations · ~32k steps/s (วัดจริง RTX 3060 6 GB)

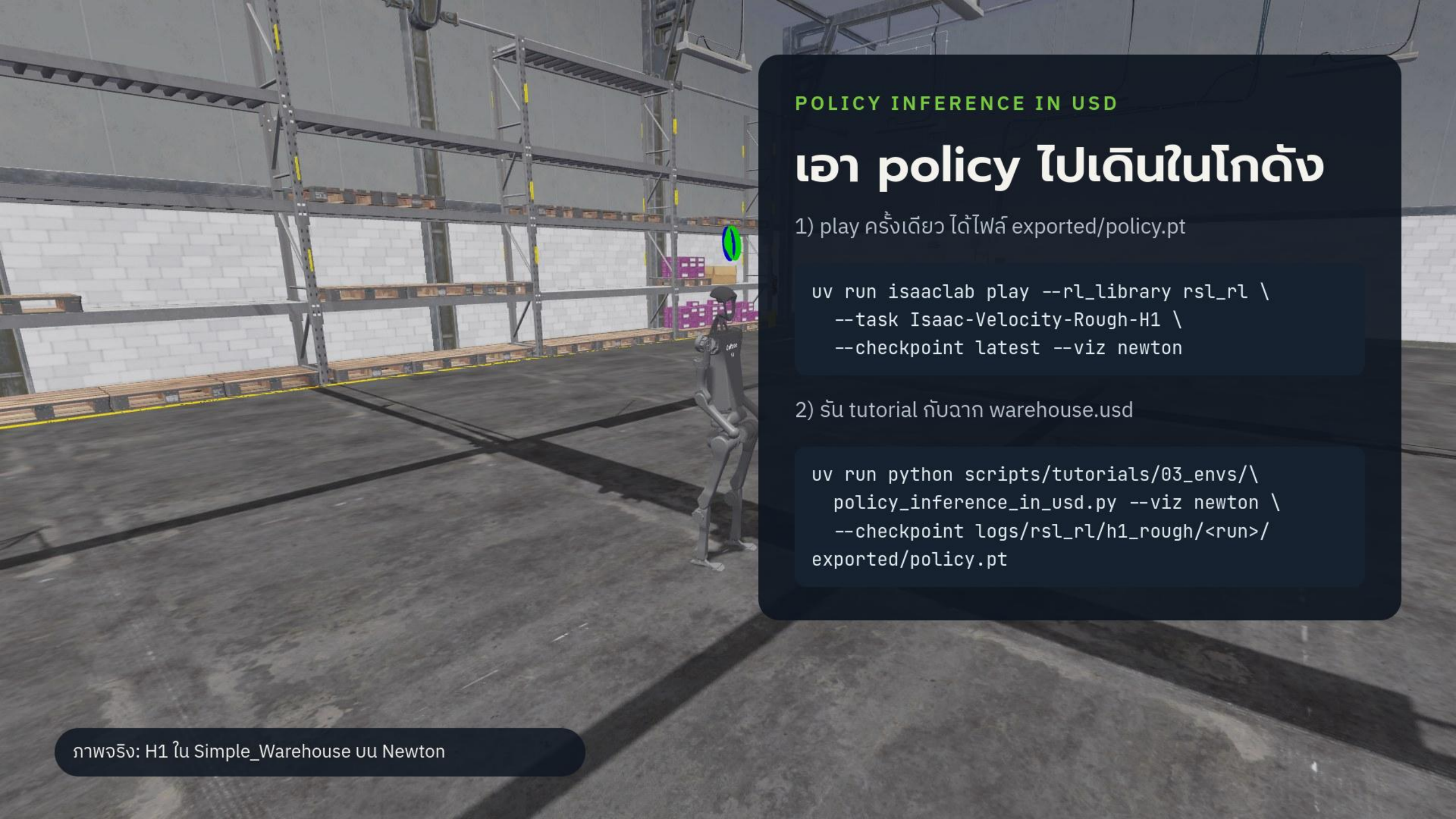
2,000

iterations × 4,096 envs คือค่าเริ่มต้น — เผื่อเวลาหลายชั่วโมง

`newton_mjwarp`

เป็น default ของ task นี้ ไม่ต้องใส่ physics=

ผลอยู่ที่ logs/rsl_rl/h1_rough/ · ไม่ต้องใส่ --headless



POLICY INFERENCE IN USD

เอา policy ไปเดินในโกดัง

1) play ครั้งเดียว ได้ไฟล์ exported/policy.pt

```
uv run isaaclab play --rl_library rsl_rl \
  --task Isaac-Velocity-Rough-H1 \
  --checkpoint latest --viz newton
```

2) รัน tutorial กับฉาก warehouse.usd

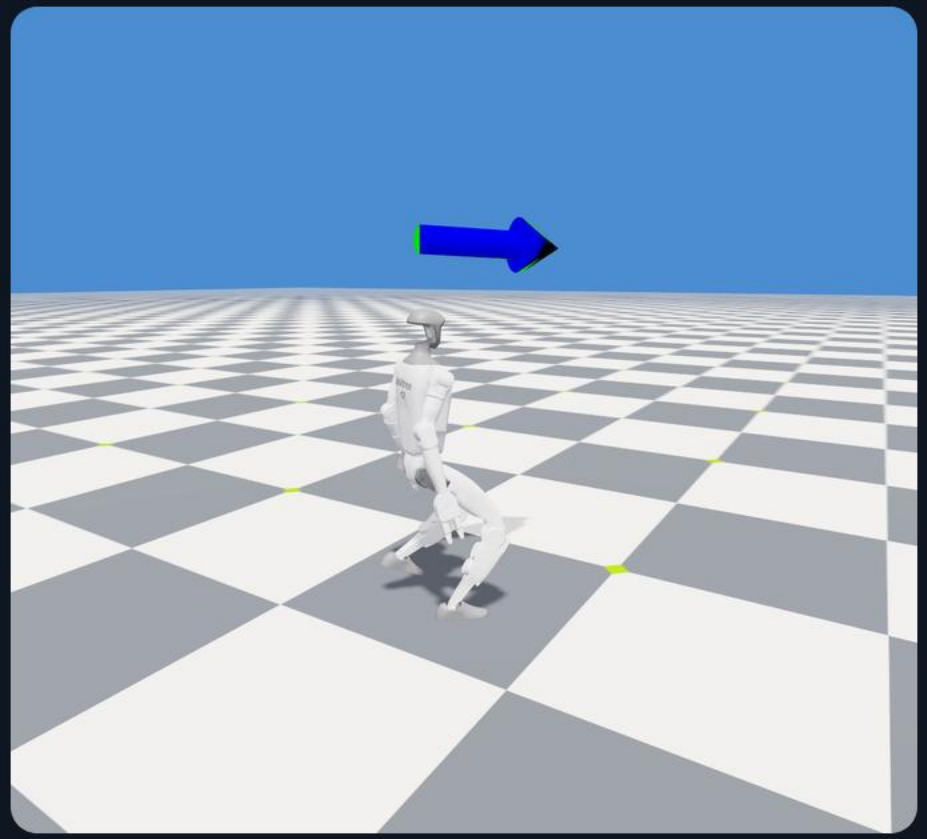
```
uv run python scripts/tutorials/03_envs/\
  policy_inference_in_usd.py --viz newton \
  --checkpoint logs/rsl_rl/h1_rough/<run>/
exported/policy.pt
```

ภาพจริง: H1 ใน Simple_Warehouse บน Newton

HANDS-ON

Control the G1 with keyboard

แกเป็นหุ่นที่ใช้... คีย์บอร์ดบังคับ



WHAT CHANGED

สคริปต์คีย์บอร์ดเดิมใช้ไม่ได้บน Newton

เดิม · Isaac Sim

- AppLauncher เปิด Omniverse Kit
- carb.input รับ keyboard event
- WASD / ลูกศรคุมหุ่น
- เขียน loop เองทั้งหมด

ใหม่ · Newton

- ไม่มี Kit → ไม่มี carb
- อ่านปุ่มจาก viewer.is_key_down()
- ใช้ I J K L U O (WASD เป็นของกล้อง)
- ใช้ pipeline ของ isaaclab play ต่อ

ไฟล์ใหม่: Week03_G1_Humanoid/code/play_interactive.py

KEY BINDINGS

กดค้าง = เดิน · ปล่อย = หยุด

U หมุนซ้าย	**I** เดินหน้า	**O** หมุนขวา
J สไลด์ซ้าย	**K** ถอยหลัง	**L** สไลด์ขวา

I · v_x = 0.8 m/s · **K** · −0.3 m/s

J / L · v_y = ±0.4 m/s

U / O · ω_z = ±1.0 rad/s

C กล้องตามหุ่น · **P** พิมพ์ตำแหน่ง · **Space** หยุดซิม · ความเร็วปรับได้ด้วย **KBD_VX**=...

WASD / QE / ลูกศร = เลื่อนกล้อง (ของ viewer)

คลิกที่หน้าต่าง viewer ก่อนกดปุ่ม

HOW IT WORKS

แทน “คำสั่งสุ่ม” ด้วย “คำสั่งจากคีย์บอร์ด”

Keyboard
`is_key_down("i")`

→

Command
`vel_command_b`

→

Observation
`velocity_commands`

→

Policy
เดินตามคำสั่ง

ตอนเทรน

command manager สุ่มคำสั่งใหม่ทุกไม่กี่วินาที

ตอนเล่น

ปิดการสุ่ม (time_left = ∞, heading off) แล้วเขียนค่าจากคีย์บอร์ดทุก step

หัวใจของ play_interactive.py

แทนแค่ loop ของ isaaclab play

```
def interactive_playback(step, *, dt, args_cli, env_cfg):
    env = _find_env(step)           # env จาก isaaclab play
    viewer = _find_viewer()         # หน้าต่าง Newton (--viz newton)
    cmd = env.command_manager.get_term("base_velocity")
    cmd.cfg.heading_command = False

    while viewer.is_running():
        key = viewer.is_key_down
        command = torch.tensor([V_X * key("i") - V_X_BACK * key("k"),
                                V_Y * (key("j") - key("l")),
                                W_Z * (key("u") - key("o"))])
        cmd.time_left[:] = 1.0e9        # ห้ามสุ่มคำสั่งใหม่
        cmd.vel_command_b[:] = command  # เขียนคำสั่งของเรา
        step()                         # policy(obs) → env.step

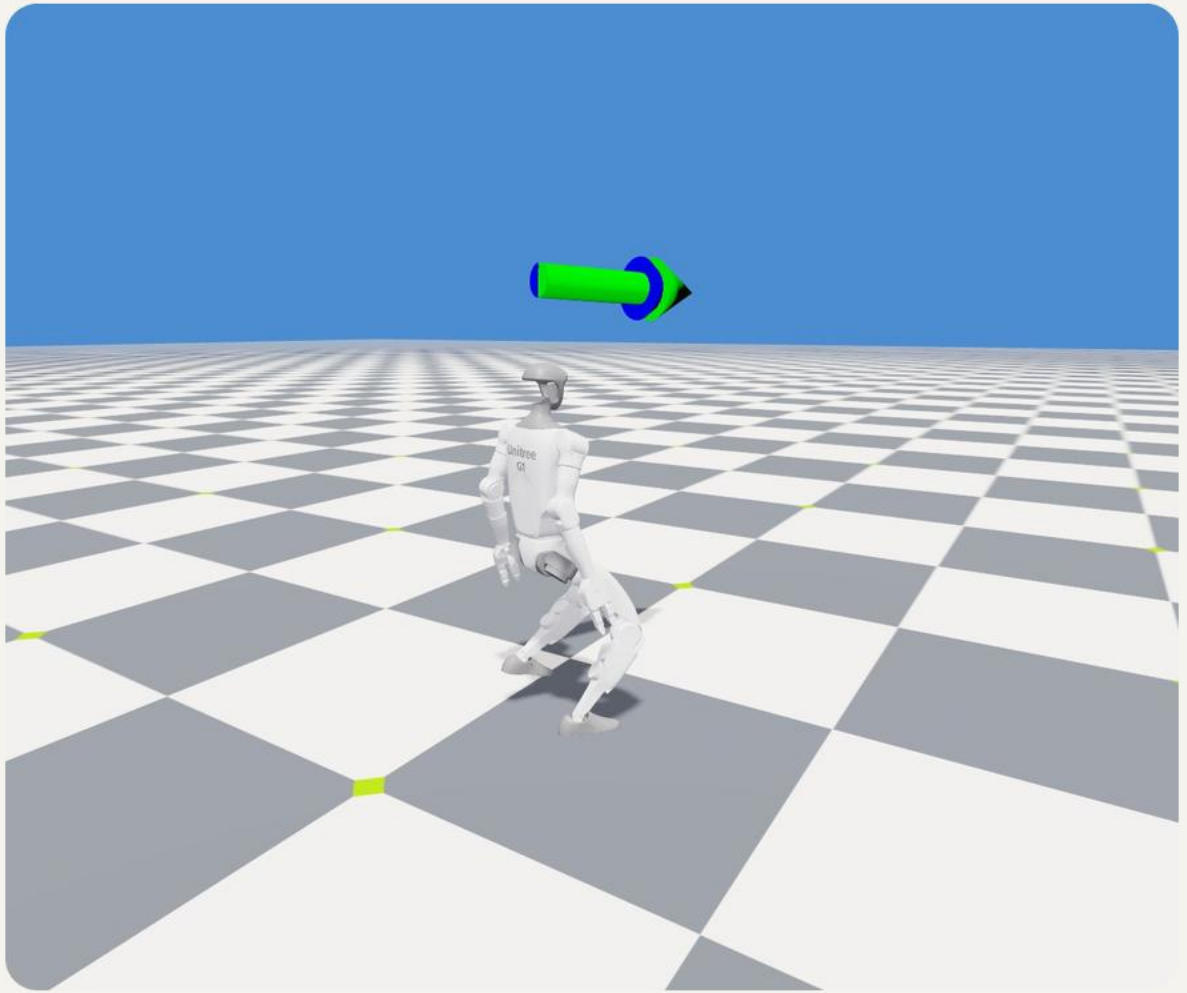
play_rsl_rl.run_playback = interactive_playback
```

RUN IT

ขับ G1 ด้วยมือเรา

```
$ cd ~/IsaacLab
$ uv run python \
  ~/Downloads/Simulation_for_Humanoid/\
  Week03_G1_Humanoid/code/
play_interactive.py \
  --task Isaac-Velocity-Flat-G1 \
  --checkpoint pretrained \
  --num_envs 1 --viz newton
```

ทดสอบแล้ว: กด I → v_x ≈ 0.7 m/s (สั่ง 0.8) · กด U → หมุนตัว · ปล่อย → หยุด



ใช้กับ H1 ได้: --task Isaac-Velocity-Rough-H1 · หรือ policy ที่เทรนเอง: --checkpoint latest

CHECKPOINT · ก่อนไปต่อ

Lab 3 ผ่านไหม?

3.1 · Pretrained humanoid

play G1 หรือ H1 ด้วย --checkpoint pretrained --viz newton

3.2 · Policy in a USD scene

ให้ H1 ยืน/เดินในโกดังด้วย policy_inference_in_usd.py

3.3 · Keyboard control

ขับ G1 ไป–กลับได้ด้วย I J K L U O · ต่อด้วย Exercises หน้าถัดไป



EXERCISES · WEEK 3

วัดผล humanoid ให้เป็นตัวเลข

4 โจทย์ · ความแม่นของการบังคับ · คำสั่งนอกช่วงที่เทรน · รัน policy นอก Isaac Lab · เทรน G1 เอง

EXERCISE 3.1 · CLOSED-LOOP ACCURACY

ขับเป็นสี่เหลี่ยม แล้วกลับมาพลาดกี่เมตร?

3.1

```
$ uv run python ~/Downloads/Simulation_for_Humanoid/Week03_G1_Humanoid/code/play_interactive.py
\
    --task Isaac-Velocity-Flat-G1 --checkpoint pretrained --num_envs 1 --viz newton
# ใหม่: กด P = พิมพ์ตำแหน่ง/ทิศของหุ่นใน terminal
```

โจทย์

1. กด **P** ที่จุดเริ่ม จด x, y, yaw
2. ขับ: เดินหน้า 2 m → เลี้ยว 90° × 4 — ลองทั้งหมุนกับที่ (U) และเลี้ยวโค้ง (I+U)
3. กด **P** ตอนจบ แล้วคำนวณ error = $\sqrt{(\Delta x^2 + \Delta y^2)}$ และ Δyaw
4. ทำ 3 รอบ หาค่าเฉลี่ย — มือคุณหรือ policy ที่พลาด?

ผลบนเครื่องทดสอบ · RTX 3060 LAPTOP 6 GB

0.70–0.79 m/s

ความเร็วจริงเมื่อสั่ง v_x = 0.8 (กด I ค้าง 5 s)

```
[POSE] x=+2.16 m y=+2.63 m yaw=+66 deg
```

ตำแหน่งวัดจากจุดกำเนิดของ env — หุ่นเกิดแบบสุ่มตำแหน่ง/ทิศ จึงต้องจดค่าเริ่มเสมอ

คิดต่อ: policy คุม **ความเร็ว** ไม่ได้คุม **ตำแหน่ง** — error สะสมเหมือน dead reckoning ถ้าอยากกลับจุดเดิมแม่น ๆ ต้องมี position controller อีกชั้นครอบ

เฉลย · EXERCISE 3.1

เลี้ยวโค้งปลอดภัยกว่าหมุนอยู่กับที่

1. เราให้ autopilot ขับแทนคน: เดินหน้าจนครบ 2 m (ดูจาก pose) → เลี้ยว 90° · ทำ 3 รอบ ด้วย G1 pretrained
2. **หมุนอยู่กับที่ (U อย่างเดียว)** ล้ม 7 ครั้งใน 3 รอบ — ส่วนใหญ่ล้มหงายหลังตอนเริ่มหมุนหลังเพิ่งเดินมา · error วัดไม่ได้จริงเพราะล้มแล้วหุ่นถูก reset
3. **เลี้ยวโค้ง (กด I + U พร้อมกัน)** ไม่ล้มเลย แต่สี่เหลี่ยมไม่ปิด: error 1.65–1.76 m · Δyaw ≈ +54° ทุกรอบ — คลาดแบบ**เป็นระบบ** (มุมโค้ง + หมุนเลยหลังปล่อยปุ่ม) ไม่ใช่สุ่ม
4. มือคนจะมี error สุ่มเพิ่มเข้าไปอีก — ถ้า 3 รอบของคุณคลาดไปทางเดียวกัน แปลว่าเป็น bias ของ policy/วิธีขับ ชดเชยได้

G1 / วิธีเลี้ยว	ล้ม	error	เวลา
หมุนอยู่กับที	7	—	—
เลี้ยวโค้ง	0	1.65–1.76 m	20 s
โค้ง · ของเรา 600 it.	0	0.51–1.02 m	43 s
โค้ง · ของเรา 1500 it.	0	1.19–1.26 m	19 s

สรุป: velocity policy ไม่มี feedback ตำแหน่ง → error สะสม · งานจริงต้องมี planner/position controller ครอบ และหลีกเลี่ยงคำสั่งที่เปลี่ยนแบบกระชาก

EXERCISE 3.2 · OUT OF DISTRIBUTION

สั่งเร็วกว่าที่เคยเทรน จะเกิดอะไร?

3.2

```
$ KBD_VX=2.0 uv run python ~/Downloads/Simulation_for_Humanoid/Week03_G1_Humanoid/code/
play_interactive.py \
    --task Isaac-Velocity-Flat-G1 --checkpoint pretrained --num_envs 1 --viz newton
# KBD_* = env var ของ play_interactive.py = ขนาดคำสั่งต่อการกดปุ่ม (ไม่ต้องแก้โค้ด)
# ค่าเริ่ม VX 0.8 · VX_BACK 0.3 (ใส่บวก สคริปต์ลบให้) · VY 0.4 m/s · WZ 1.0 rad/s
# G1 Flat เทรนด้วย lin_vel_x ∈ [0.0, 1.0] → 2.0 m/s อยู่นอกช่วง
```

สั่ง v_x	วัดได้	ผล
0.8 m/s (ในช่วง)	0.70–0.79 m/s	ไม่ล้ม
2.0 m/s (นอกช่วง)	1.66–1.72 m/s	ไม่ล้มใน 5 s

โจทย์

1. ลอง **KBD_VX=3.0** และ **KBD_WZ=3.0** — เริ่มล้มที่ค่าไหน?
2. ตั้ง **KBD_VX_BACK=0.8** แล้วกด **K** (ถอยหลัง) — G1 Flat ไม่เคยเทรนคำสั่งติดลบ
3. หาช่วงที่เทรนใน **core/velocity/config/g1/flat_env_cfg.py**

คิดต่อ: policy ไม่ได้ “ทำตามคำสั่ง” — มันทำในสิ่งที่เคยได้ reward ระหว่างเทรน ก่อนส่งคำสั่งให้หุ่นจริง ต้อง clamp ให้อยู่ในช่วงที่เทรนมา

เฉลย · EXERCISE 3.2

นอกช่วงที่เทรน = ไม่มีอะไรรับประกัน

1. **KBD_VX=3.0**: ได้ ~1.7 m/s แต่ล้ม 6 ครั้งใน 12 s · **KBD_WZ=3.0**: หมุนได้ ~1.3 rad/s ล้ม 3 ครั้ง — ที่ 2.0 m/s ยังไม่ล้ม จุดเริ่มพังอยู่ระหว่าง 2–3 m/s
2. **KBD_VX_BACK=0.8** (กด K): ล้ม 21 ครั้งใน 6 รอบ · 0.5 ล้ม 3 · 0.3 ไม่ล้มแต่ถอยได้แค่ −0.15 m/s — ไม่เคยเทรนคำสั่งติดลบ สคริปต์จึงตั้งค่าถอยเริ่มต้นไว้ 0.3
3. ช่วงที่เทรน (flat_env_cfg.py): **lin_vel_x (0.0, 1.0) · lin_vel_y (−0.5, 0.5) · ang_vel_z (−1.0, 1.0)** · แต่ G1 เทรนด้วย heading_command: wz = 0.5 × (ทิศเป้า − ทิศปัจจุบัน) แล้วจำกัด ±1 — ไม่ได้สุ่มตรง ๆ

สั่ง v_x	ได้จริง	ล้ม
0.8 (ใน)	0.73	0
2.0	1.66–1.72	0
3.0	~1.7	6
−0.8 (K)	—	21*

G1 Flat pretrained · กดค้าง 0.8 → 20 s · 2.0 → 5 s · อื่น ๆ 12 s

* กด K ค้าง 6 s จากท่ายืน × 6 รอบ

สรุป: ก่อนส่งคำสั่งให้หุ่นจริง clamp ให้อยู่ในช่วงที่เทรน — และขยายช่วงตอนเทรนถ้าต้องการความสามารถนั้น

EXERCISE 3.3 · DEPLOY-READY EXPORT

รัน policy โดยไม่มี Isaac Lab

3.3

```
$ uv run python ~/Downloads/Simulation_for_Humanoid/Week03_G1_Humanoid/code/run_onnx_policy.py \
    logs/rsl_rl/<task>/<run>/exported
# ใช้แค่ onnxruntime + numpy — แบบเดียวกับที่จะใช้บนคอมพิวเตอร์ของหุ่นจริง
```

ผลบนเครื่องทดสอบ · RTX 3060 LAPTOP 6 GB

```
input  obs     [1, 123]
output actions [1, 37]
max |onnx - torchscript| = 9.5e-07
onnxruntime CPU: 0.014 ms per call
```

G1 Flat pretrained · ONNX กับ TorchScript ให้ผลตรงกัน · เร็วกว่ารอบควบคุม 20 ms (50 Hz) กว่าพันเท่า

โจทย์

1. export จาก run ของคุณเอง: รัน **isaaclab play** หนึ่งครั้ง
2. 123 = อะไรบ้าง? เทียบกับตาราง obs ตอนเริ่มเทรน
3. 37 actions = มุมเป้าหมายของข้อต่อไหนบ้าง? ดู actuator ของ G1 ใน **isaaclab_assets/robots/unitree.py**

คิดต่อ: เวลาเอาขึ้นหุ่นจริง ส่วนที่ยากไม่ใช่ network แต่คือการสร้าง obs 123 ค่า **ให้เรียงและ scale ตรงกับตอนเทรน** — ดูลำดับจากตาราง Observation Manager

เฉลย · EXERCISE 3.3

obs 123 ค่า = 7 กลุ่ม เรียงตามลำดับนี้เท่านั้น

1. รัน **isaaclab play** หนึ่งครั้ง → **logs/rsl_rl/g1_flat/<run>/exported/** มี policy.onnx (+ .onnx.data) และ policy.pt
2. 123 = base_lin_vel 3 + base_ang_vel 3 + projected_gravity 3 + velocity_commands 3 + joint_pos 37 + joint_vel 37 + actions (ก่อนหน้า) 37
3. 37 actions = มุมเป้าหมายของข้อต่อ: ขา 12 (hip ×3, knee, ankle ×2 ต่อข้าง) · torso 1 · แขน 10 (shoulder ×3, elbow ×2 ต่อข้าง) · มือ 14 (นิ้ว 7 ต่อข้าง)

```
# ลำดับใน obs vector
[0:3]    base_lin_vel
[3:6]    base_ang_vel
[6:9]    projected_gravity
[9:12]   velocity_commands
[12:49]  joint_pos (rel)
[49:86]  joint_vel (rel)
[86:123] last actions
```

สรุป: ตอน deploy ต้องสร้าง obs ให้เรียง/หน่วย/scale ตรงตอนเทรนทุกตัว — ผิดหนึ่งตัว policy ก็ทำงานผิดโดยไม่ error

EXERCISE 3.4 · TRAIN YOUR OWN HUMANOID

เทรน G1 เอง เทียบกับ pretrained

3.4

```
$ uv run isaaclab train --rl_library rsl_rl --task Isaac-Velocity-Flat-G1 \
    --num_envs 1024 --max_iterations 600 --run_name ex34_600  # 9 นาที
# ค่าเริ่มของ G1 Flat = 4,096 envs × 1,500 it · ที่นี่ 1,024 × 600 → ข้อมูลแค่ ~1/10 ของ pretrained (เทียบไม่ยุติธรรม)
$ … play_interactive.py --task Isaac-Velocity-Flat-G1 --checkpoint latest …
```

iter.	ep. len	reward	สังเกต
100	54	−5.1	ล้มใน ~1 s
250	162	−6.5	เริ่มยืนได้
300	766	−11.4	ยืนได้ ยังไม่เดิน
450	969	−2.9	เริ่มเดินตามคำสั่ง
600	968	+5.3	สั่ง 0.8 → ได้ ~0.6 m/s

โจทย์

1. ขับ policy ของคุณ เทียบกับ **pretrained** (0.70–0.79 m/s)
2. เพิ่มเป็น **--max_iterations 1500** (ค่าเริ่มต้นของ G1 Flat) แล้ววัดความเร็วซ้ำ

คิดต่อ: reward **ลดลง** ช่วงที่หุ่นเริ่มยืนได้ — ยืนนานขึ้นจึงโดน penalty สะสมมากขึ้น ดู episode length คู่กับ reward เสมอ

เฉลย · EXERCISE 3.4

เทรนนานขึ้น = ใกล้ pretrained ขึ้น

1. สั่ง 0.8 m/s (กด I ค้าง): 600 it. ได้ 0.59–0.63 · **1,500 it. ได้ 0.72** · pretrained 0.73 — ที่ 1,500 iterations ความเร็วเทียบเท่า pretrained แล้ว
2. Exercise 3.1 ซ้ำ: 600 it. error ดูเล็ก (0.51–1.02 m) แต่ใช้ 43 s เพราะเดิน/เลี้ยวช้า · 1,500 it. ใช้ 19 s และคลาดแบบเป็นระบบเหมือน pretrained (1.19–1.26 m · Δyaw ≈ +31°) — **error ต่ำไม่ได้แปลว่าดีกว่า ต้องดูเวลาคู่กัน**
3. 1,500 × 1,024 envs ใช้ 23 นาที (28k steps/s) · ep. length เต็มตั้งแต่ ~750 iterations แต่ reward ยังขึ้นต่อถึงจบ (7.0 → 17.0) = ยังเรียนเรื่องคุณภาพการเดินอยู่

iter.	ep. len	reward
150	56	−4.9
300	573	−11.2
600	989	+2.8
900	971	+9.6
1,200	996	+14.5
1,500	986	+17.0

สรุป: humanoid ต้องเทรนนานกว่าหุ่นสี่ขามาก — วัดผลด้วยงานจริง (ความเร็ว, เวลา, error) ไม่ใช่แค่ reward